

Event-Driven Models

Task-1. Create an S3 bucket named 'music-uploads-demo' and upload a sample MP3 file to it using the AWS Management Console.

1. Open [AWS Management Console](#) and sign in.
2. Search for **S3** and open the S3 service.
3. Click **Create bucket**.
4. Enter the bucket name:
music-uploads-demo
5. Select your desired AWS Region.
6. Leave the other settings at their defaults for this exercise.
7. Click **Create bucket**.
8. Open the bucket → **Upload** → **Add files**.
9. Select a sample .mp3 file from your computer and click **Upload**.

The screenshot displays the AWS Management Console interface for the S3 service. At the top, the navigation bar shows the AWS logo, a search bar, and the user's profile (Mehta Darshit). The main content area is titled 'Amazon S3' and features a 'Summary' section with a 'Destination' of 's3://music-uploads-demo'. The 'Summary' section indicates that 1 file (716.5 KB) has been successfully uploaded (100.00%) and 0 files (0 B) have failed (0%). Below the summary, the 'Files and folders' tab is selected, showing a list of files. The list contains one file named 'file_example_MP3_700K...' with a size of 716.5 KB, type 'audio/mpeg', and a status of 'Succeeded'. The bottom of the screen shows the Windows taskbar with various application icons and the system clock indicating 12:35 PM on 06-Oct-26.

Name	Folder	Type	Size	Status	Error
file_example_MP3_700K...	-	audio/mpeg	716.5 KB	Succeeded	-

Tasks-2. Write an AWS Lambda function in Python called `log_upload_event` that logs the filename and upload timestamp whenever a new file is added to your 'music-uploads-demo' S3 bucket.

Open AWS Lambda → Create function → Author from scratch.

Use:

- **Function name:** `log_upload_event`
- **Runtime:** Python 3.x

```
import urllib.parse
```

```
from datetime import datetime, timezone
```

```
def lambda_handler(event, context):
```

```
    for record in event.get("Records", []):
```

```
        bucket_name = record["s3"]["bucket"]["name"]
```

```
        object_key = urllib.parse.unquote_plus(record["s3"]["object"]["key"])
```

```
        upload_timestamp = datetime.now(timezone.utc).isoformat()
```

```
        print(f"Filename: {object_key}")
```

```
        print(f"Bucket: {bucket_name}")
```

```
        print(f"Upload timestamp: {upload_timestamp}")
```

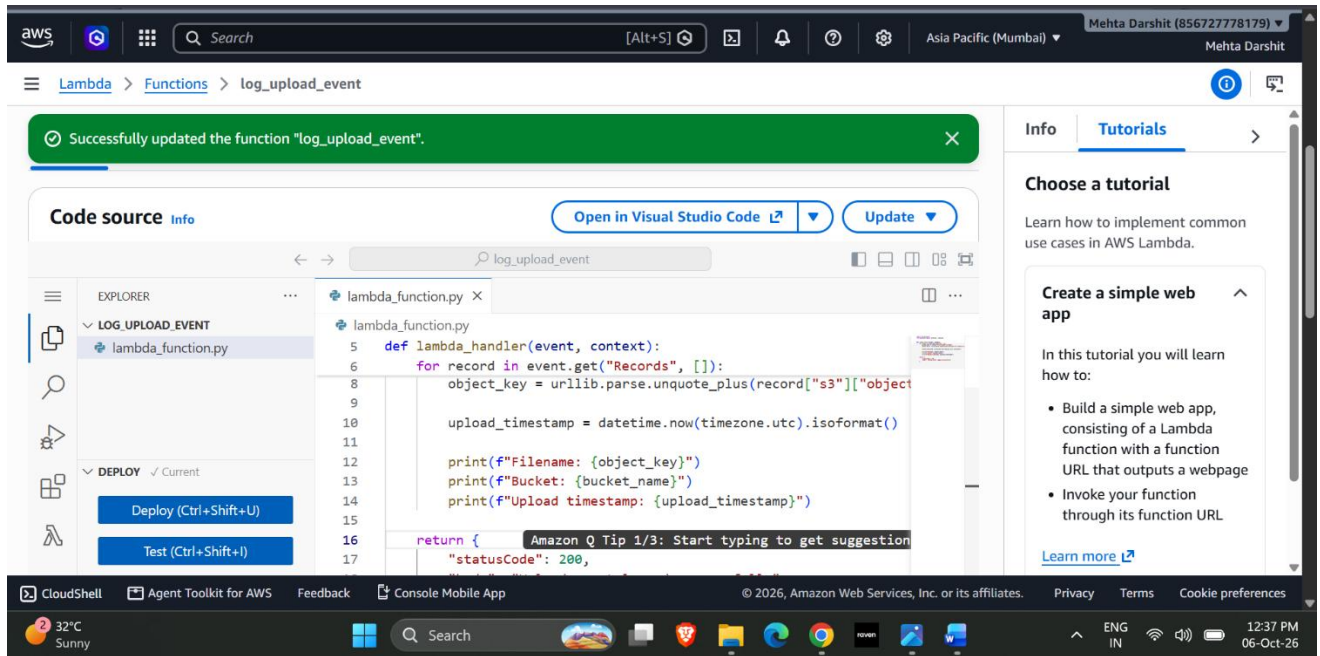
```
    return {
```

```
        "statusCode": 200,
```

```
        "body": "Upload event logged successfully"
```

```
    }
```

Click Deploy.



Tasks-3. Configure an event trigger so that your log_upload_event Lambda function is automatically invoked every time a new file is uploaded to the 'music-uploads-demo' S3 bucket.

❓ Open log_upload_event.

❓ On the function's **Configuration** or **Function overview** area, choose **Add trigger**.

❓ Select **S3**.

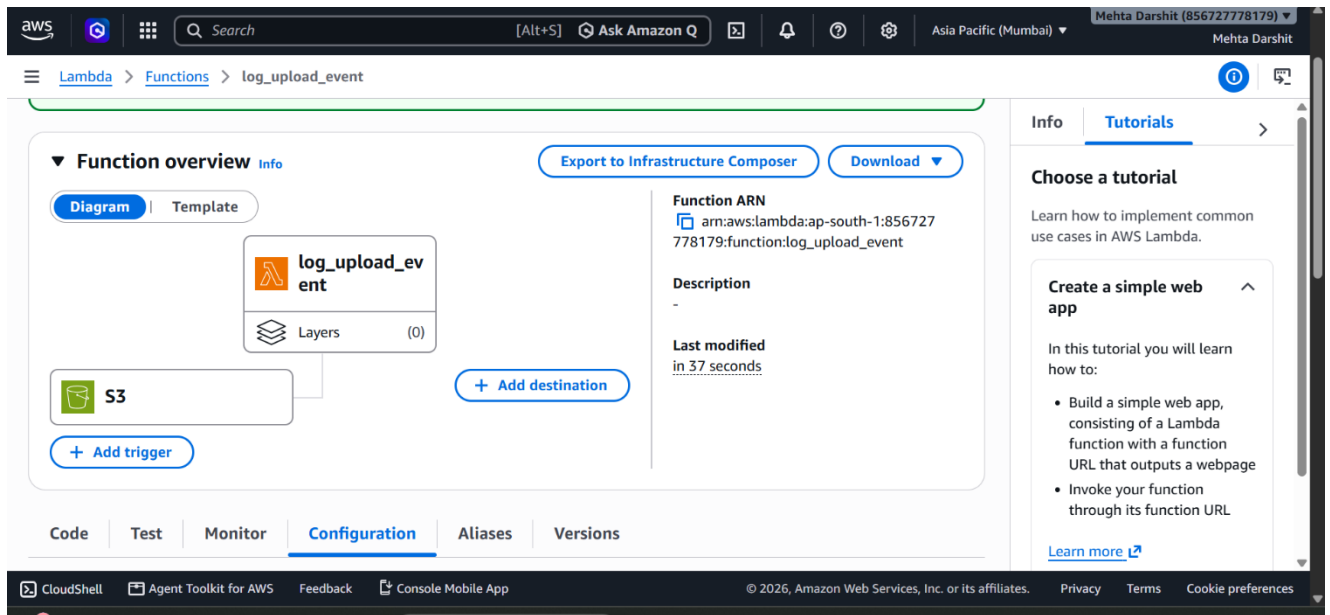
❓ Select your bucket:
music-uploads-demo

❓ For the event type, select **All object create events**.

❓ Optionally specify .mp3 as a suffix filter if you want the Lambda to respond only to MP3 uploads.

❓ Confirm the acknowledgement that using this configuration grants the necessary S3 invocation permission.

❓ Click **Add**.



Tasks-4. Simulate uploading a new file to your S3 bucket and verify that your Lambda function runs by checking the logs in AWS CloudWatch. Look for the Lambda function's log group in CloudWatch to see the output for each upload event.

Upload another MP3 with a different filename, for example:

test-song.mp3

Wait a few seconds, then:

1. Open **Lambda** → **log_upload_event**.
2. Select **Monitor**.
3. Choose **View CloudWatch logs**.
4. Open the latest **log stream**.

The screenshot shows the AWS CloudWatch console interface. The left sidebar contains navigation links for CloudWatch, Log management, and various AWS services. The main panel displays 'Log events' for a specific log group. A search bar at the top allows filtering events. Below the search bar, a table lists log events with columns for Timestamp and Message. The events show a sequence of operations: START, filename upload, bucket specification, upload timestamp, END, and a REPORT. The console also includes buttons for 'Actions', 'Start tailing', and 'Display'.

Timestamp	Message
2026-10-06T07:20:49.567Z	START RequestId: e6cc032a-67c2-4d72-8cb3-fca90f5037bb Version: \$LATEST
2026-10-06T07:20:49.567Z	Filename: file_example_MP3_1MG.mp3
2026-10-06T07:20:49.567Z	Bucket: music-uploads-demo
2026-10-06T07:20:49.567Z	Upload timestamp: 2026-10-06T07:20:49.567610+00:00
2026-10-06T07:20:49.574Z	END RequestId: e6cc032a-67c2-4d72-8cb3-fca90f5037bb
2026-10-06T07:20:49.574Z	REPORT RequestId: e6cc032a-67c2-4d72-8cb3-fca90f5037bb Duration: 1.97 ms Billed Duration: 2.00 ms

Tasks-5. Use ChatGPT or GitHub Copilot to suggest an additional event-driven use case for S3 and Lambda (other than logging uploads), and describe in 2-3 lines how you would implement it.

Use case: Automatically create thumbnails

When an image is uploaded to an S3 bucket, an S3 event can trigger a Lambda function. The Lambda function can download the image, create a smaller thumbnail, and save the thumbnail to another S3 folder/bucket.

For example:

Image upload → S3 event → Lambda → resize image → /thumbnails/ in S3